Лабораторная работа №8

Паттерны проектирования в Python

1. Цель работы

Изучить порождающие и поведенческие паттерны проектирования, применив Factory, Strategy и Observer совместно с Repository.

2. Постановка индивидуального задания

Вариант 12 — «Курьерская служба». Применить паттерны проектирования для построения расширяемой системы.

Программа должна:

– Создать виды грузов: документы, посылка и крупный груз через фабрику.

– Реализовать три стратегии цены: стандартную, срочную и экономичную.

– Добавить уведомление клиента и курьера через Observer и отделить хранение в Repository.

3. Краткие теоретические сведения

Паттерн проектирования — типовая структура решения повторяющейся архитектурной проблемы. Порождающий Factory сосредоточивает создание объектов одного семейства, а поведенческий Strategy заменяет ветвления взаимозаменяемыми алгоритмами.

Observer реализует подписку компонентов на изменения состояния. Наблюдатели получают уведомления от сервиса, который не зависит от конкретного вида получателя. Repository выделяет слой хранения и поиска объектов.

В проекте стандартный тариф: 100 + 20 × расстояние + 10 × вес (руб.). Срочный тариф умножает полученную стоимость на 1,5, экономичный — на 0,8. Это учебные коэффициенты, определённые в реализации.

4. Исходный код программы

Файл before.py

```
# До паттернов: выбор груза, тарифа и уведомлений смешан в одной функции.
def delivery_cost(kind, tariff, distance, weight, status):
    if kind == "документы":
        item = {"kind": "документы", "weight": weight}
    elif kind == "посылка":
        item = {"kind": "посылка", "weight": weight}
    elif kind == "крупный груз":
        item = {"kind": "крупный груз", "weight": weight}
    else:
        raise ValueError("Неизвестный груз")
    cost = 100 + distance * 20 + item["weight"] * 10
    if tariff == "срочная":
        cost *= 1.5
    elif tariff == "экономичная":
        cost *= 0.8
    elif tariff != "стандартная":
        raise ValueError("Неизвестный тариф")
    print("Клиент:", status)
    print("Курьер:", status)
    return cost
```

Файл models.py

```
from dataclasses import dataclass
from enum import Enum


class Status(str, Enum):
    CREATED = "создана"
    ACCEPTED = "принята"
    MOVING = "в пути"
    DELIVERED = "доставлена"
    CANCELLED = "отменена"


@dataclass
class Delivery:
    id: int
    address: str
    weight: float
    distance: float
    courier: str = ""
    status: Status = Status.CREATED
```

Файл repository.py

```
from typing import Protocol
from models import Delivery


class Reader(Protocol):
    def get(self, id: int) -> Delivery: ...
    def get_all(self) -> list[Delivery]: ...
```

```
class Writer(Protocol):
    def add(self, delivery: Delivery) -> None: ...
    def delete(self, id: int) -> None: ...


class Repository(Reader, Writer, Protocol):
    pass


class MemoryRepository:
    def __init__(self) -> None:
        self.items: dict[int, Delivery] = {}

    def add(self, delivery: Delivery) -> None:
        if delivery.id in self.items:
            raise ValueError("id уже существует")
        self.items[delivery.id] = delivery

    def get(self, id: int) -> Delivery:
        if id not in self.items:
            raise ValueError("Доставка не найдена")
        return self.items[id]

    def get_all(self) -> list[Delivery]:
        return list(self.items.values())

    def delete(self, id: int) -> None:
        self.get(id)
        del self.items[id]
```

Файл patterns.py

```
from dataclasses import dataclass
from typing import Protocol
from models import Delivery


@dataclass
class Documents(Delivery):
    kind: str = "документы"


@dataclass
class Parcel(Delivery):
    kind: str = "посылка"


@dataclass
class LargeCargo(Delivery):
    kind: str = "крупный груз"


class DeliveryFactory:
    types = {"документы": Documents, "посылка": Parcel, "крупный груз": LargeCargo}
```

```
    @staticmethod
    def create(kind: str, id: int, address: str, weight: float, distance: float) -> Delivery:
        if kind not in DeliveryFactory.types:
            raise ValueError("Неизвестный вид груза")
        return DeliveryFactory.types[kind](id, address, weight, distance)


class Pricing(Protocol):
    def calculate(self, delivery: Delivery) -> float: ...


class StandardPricing:
    def calculate(self, delivery: Delivery) -> float:
        return 100 + delivery.distance * 20 + delivery.weight * 10


class UrgentPricing:
    def calculate(self, delivery: Delivery) -> float:
        return StandardPricing().calculate(delivery) * 1.5


class EconomyPricing:
    def calculate(self, delivery: Delivery) -> float:
        return StandardPricing().calculate(delivery) * 0.8


class Observer(Protocol):
    def update(self, delivery: Delivery) -> None: ...


class ClientObserver:
    def update(self, delivery: Delivery) -> None:
        print(f"Клиент: доставка {delivery.id} — {delivery.status.value}")


class CourierObserver:
    def update(self, delivery: Delivery) -> None:
        print(f"Курьер {delivery.courier}: доставка {delivery.id} — {delivery.status.value}")
```

Файл services.py

```
from models import Delivery, Status
from repository import Repository
from patterns import Pricing, Observer


class DeliveryService:
    def __init__(self, repository: Repository, pricing: Pricing) -> None:
        self.repository = repository
        self.pricing = pricing
        self.observers: list[Observer] = []

    def subscribe(self, observer: Observer) -> None:
        self.observers.append(observer)
```

```
    def create(self, delivery: Delivery) -> None:
      if delivery.id <= 0 or not delivery.address or delivery.weight <= 0 or delivery.distance < 0:
          raise ValueError("Некорректная доставка")
       self.repository.add(delivery)

    def cost(self, id: int) -> float:
       return self.pricing.calculate(self.repository.get(id))

    def change_status(self, id: int, status: str) -> None:
       delivery = self.repository.get(id)
       transitions = {"создана": ["принята", "отменена"],
                      "принята": ["в пути", "отменена"],
                      "в пути": ["доставлена", "отменена"]}
       if status not in transitions.get(delivery.status, []):
          raise ValueError("Недопустимый переход")
       delivery.status = Status(status)
       for observer in self.observers:
          observer.update(delivery)
```

Файл main.py

```
from repository import MemoryRepository
from services import DeliveryService
from patterns import (DeliveryFactory, StandardPricing, UrgentPricing, EconomyPricing,
                      ClientObserver, CourierObserver)


def main() -> None:
    repository = MemoryRepository()
    service = DeliveryService(repository, StandardPricing())
    service.subscribe(ClientObserver())
    service.subscribe(CourierObserver())
    for id, kind in enumerate(DeliveryFactory.types, 1):
        delivery = DeliveryFactory.create(kind, id, "ул. Пушкина, 10", id, 5)
        delivery.courier = "Максим"
        service.create(delivery)
        print(delivery)
    for pricing in (StandardPricing(), UrgentPricing(), EconomyPricing()):
        service.pricing = pricing
        print(type(pricing).__name__, service.cost(1))
    service.change_status(1, "принята")
    service.change_status(1, "в пути")


if __name__ == "__main__":
    main()
```

5. Примеры работы программы в виде снимков экрана

На рисунке 8.1 приведён фрагмент контрольного запуска применения паттернов Factory, Strategy и Observer.

```
python main.py
Documents(id=1, address='ул. Пушкина, 10', weight=1, distance=5,
courier='Максим', status=<Status.CREATED: 'создана'>, kind='документы')
Parcel(id=2, address='ул. Пушкина, 10', weight=2, distance=5,
courier='Максим', status=<Status.CREATED: 'создана'>, kind='посылка')
LargeCargo(id=3, address='ул. Пушкина, 10', weight=3, distance=5,
courier='Максим', status=<Status.CREATED: 'создана'>, kind='крупный груз')
StandardPricing 210
UrgentPricing 315.0
EconomyPricing 168.0
Клиент: доставка 1 — принята
Курьер Максим: доставка 1 — принята
Клиент: доставка 1 — в пути
Курьер Максим: доставка 1 — в пути
```

Рисунок 8.1 – Результат контрольного запуска программы

Автоматическое тестирование выполнено с помощью pytest. Все 7 тестов прошли успешно (рисунок 8.2).

```
pytest -q
.......
[100%]
7 passed in 0.03s
```

Рисунок 8.2 – Результат автоматического тестирования

6. Вывод

Внедрены Factory, Strategy, Observer и Repository. Создание грузов, выбор тарифа и уведомление подписчиков выполняются независимыми компонентами. Все 7 тестов успешно пройдены.

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	